

Contest Platform with Live Leaderboard

Database Implementation & Core Functionalities (MongoDB + Redis)

Team	Upayan Mazumder (24BDS0367), Aditya Rawat (24BCE2992), Shloak Sinha (24BDS0378)
Course	BCSE302P – Database Systems Lab
Component	Review 2 – Database Implementation & Core Functionalities
Repository	github.com/upayanmazumder/rankstack

1. Database Implementation

We used MongoDB as the primary database for this project. The data used in a contest platform, such as problems that have different fields depending on whether they are MCQ, coding or subjective questions, fits more naturally into a document model than into fixed relational tables. Redis is used alongside MongoDB for data that changes frequently or does not need to be stored permanently, such as leaderboard scores and login sessions.

The project uses five collections: users, teams, contests, problems and submissions.

- A user document stores the name, email, hashed password, role and total score.
- A team document stores the team name, the list of member user ids, and the team's total score.
- A contest document stores the title, start and end time, current status, and the list of problem ids and participants.
- A problem document stores which contest it belongs to, its type (mcq, coding or subjective), difficulty, points, and how many times it has been attempted.
- A submission document stores the contest, the problem, who submitted it, its status and score.

A contest can have multiple problems and multiple participants, and a participant can be either a user or a team. Every submission is linked to the problem it was made for and to the participant who made it. A team also keeps a list of its member users, so a user can belong to a team and submit either individually or as part of that team.

```

contests -----> problems
contests -----> participants (users / teams)
participants -----> submissions <----- problems
teams -----> users (team members)

```

Every collection has a validation rule attached to it using MongoDB's `$jsonSchema`, so the required fields and their types are checked when a document is inserted or updated, instead of relying only on checks inside the API code. For example, the validator on the problems collection requires the `type` field to be one of mcq, coding or subjective. These validators, along with the indexes described in Section 3, are created by `scripts/init_db.py`, which can be run again without creating duplicate collections.

`scripts/seed.py` was used to load sample data into all five collections. Review 2 asks for at least 50 sample records, and our seed script currently inserts 88:

Collection	Records
users	15
teams	5
contests	5
problems	15
submissions	48
Total	88

The records are related correctly to each other – every submission in the seed data points to a problem, a contest and a participant that actually exist, so the aggregation results shown later in this document are computed from consistent data.

2. CRUD Operations

For demonstrating CRUD operations, we used the users collection. Creating a user sends a POST request with the name, email and password, and the API returns the created user along with a generated id. The same id is then used to read, update and delete that user.

```
POST /users {"name":"Doc User","email":"doc@rankstack.io","password":"secret123"}
-> 201 {"id":"6ab3f9a5...", "name":"Doc User", "role":"participant", "totalScore":0}

GET    /users/6ab3f9a5...    -> 200 {"id":"6ab3f9a5...", "name":"Doc User", ...}
PATCH /users/6ab3f9a5...    -> 200 {"name":"Doc User (edited)", ...}
DELETE /users/6ab3f9a5...    -> 204
```

The other four collections (teams, contests, problems and submissions) support the same four operations in the same way. In total, the API has 33 REST endpoints across these five collections and the session routes, but they all follow the create-read-update-delete pattern shown above.

3. Advanced NoSQL Features

3.1 Aggregation Pipelines

MongoDB's aggregation framework was used for calculations that a plain `find()` query cannot produce directly, since these need totals or comparisons across multiple documents.

- The **leaderboard** pipeline groups submissions by participant, adds up their scores and ranks them, so the standings for a contest can be returned directly.
- The **average score** pipeline groups problems by difficulty and works out the average score of submissions made to problems in that group.
- The **submission status** pipeline counts submissions in each contest by their status (pending, correct, incorrect, partial).
- The **distinct problems solved** pipeline works out how many different problems each user has solved correctly, since the same user may submit to a problem more than once.
- The **team performance** pipeline uses `$lookup` to bring together a team's members and their submissions, so a team's total score, submission count and member count come back as one document.

Pipeline	Purpose	Endpoint
Leaderboard	total score and rank per participant, for one contest	/aggregations/leaderboard/{id}
Average score by difficulty	mean score of submissions, grouped by problem difficulty	/aggregations/avg-score-by-difficulty
Submission status by contest	submission count per status, per contest	/aggregations/submission-status-by-contest
Distinct problems solved	number of unique problems solved by each user	/aggregations/problems-solved-per-user
Team performance	combined score and submission count for a team's members	/aggregations/team-performance

3.2 Indexing

Indexes were added on fields that are queried often, or that need to stay unique, so MongoDB does not have to scan every document in a collection to answer a query.

- A **unique index on users.email** stops two accounts from registering with the same email address and speeds up login lookups.
- An **index on problems.contestId** is used whenever a contest's problem list is fetched.
- A **compound index on submissions (contestId, submittedBy.refId)** is used to fetch one participant's submissions within a specific contest, which comes up whenever a user's own submission history is shown.
- An **index on submissions.problemId** supports the distinct-problems-solved aggregation and other problem-level statistics.
- An **index on teams.memberIds** is used to look up which team a user belongs to.
- An **index on contests.status** is used to filter contests that are live or upcoming.

We checked two of these queries with `explain()` while testing, and the winning plan came back as IXSCAN with the expected index name instead of a full collection scan.

3.3 Transactions

When a submission is created, both the submission document and the attempt count on the corresponding problem need to be updated. If only one of these writes went through, for example because of a crash between the two operations, the data would become inconsistent. To avoid this, the two writes are placed inside a single transaction using a MongoDB session, so either both of them happen or neither does.

The same idea is used in three other places. Updating a submission's status also updates the score of the submitter (a user or a team) in the same transaction. Adding or removing a member from a team updates both the team's member list and a reference stored on the user's own document, and this is also done together as one transaction.

MongoDB only supports multi-document transactions when it is running as a replica set. This is why the project runs MongoDB as a single-node replica set (`rs0`) even during local development, instead of as a standalone instance.

3.4 Redis

We used Redis for data that changes often or does not need to be kept permanently, alongside MongoDB.

- **Leaderboard** – each contest has a Redis sorted set. A participant's score is updated with `ZINCRBY` whenever their submission score changes, and the top participants are read back with `ZREVRANGE`.
- **Sessions** – after logging in, a session key is stored in Redis with a TTL of one hour (3600 seconds), so old sessions expire on their own without a separate cleanup job.
- **Rate limiting** – a counter is kept per user for a 60 second window. Once a user crosses 10 submissions in that window, further submissions are blocked until the window resets.

4. Intermediate Demonstration

During the intermediate review, the project can be demonstrated in the following order:

1. Check the MongoDB and Redis connection
2. Demonstrate CRUD operations on a user
3. Show an indexed query and its `explain()` output
4. Demonstrate the compound index on submissions
5. Demonstrate a transaction by creating a submission
6. Run all five aggregation pipelines
7. Show the Redis leaderboard, session TTL and rate limiting

This flow is scripted in `scripts/demo.py`, which runs against the already-seeded database and prints the result of each step as it happens. The submission it creates for the transaction step is removed again at the end, so the script can be run more than once during the review without leaving extra data behind.

5. Checklist

Criterion	Marks	Where to look	Status
Database implementation	2	Section 1, <code>scripts/init_db.py</code>	Completed
CRUD operations	3	Section 2, <code>api/routes/</code>	Completed
Advanced NoSQL features	5	Section 3, <code>api/aggregations/</code> , <code>api/services.py</code>	Completed
50+ sample records	–	88 records, <code>scripts/seed.py</code>	Completed
Intermediate demonstration	–	<code>scripts/demo.py</code>	Completed
Source code repository	–	github.com/upayanmazumder/rankstack	Completed

6. Running the Project

The following commands set up and run the project locally:

```
make up          # MongoDB replica set + Redis in Docker
make init-db     # collections, $jsonSchema validators, indexes
make seed        # 88 sample records
make demo        # the walkthrough described in Section 4
make api         # FastAPI on http://localhost:8000/docs
```